Swap

# Quickstart: Swap tokens on a blockchain

Copy page

Use App Kit to swap one token for another on the same blockchain

This quickstart walks you through how to use App Kit's **Swap** capability to swap tokens on the same blockchain. The example swaps USDC for EURC on Arc Testnet, but you can use other **supported tokens or blockchains**.

## Prerequisites

Before you begin, ensure that you've:

- Installed **Node.js v22+**.
- Created an EVM wallet using a wallet provider such as **MetaMask** and added **Arc Testnet**.
- Funded the wallet with testnet USDC and native tokens for fees from the **Circle Faucet**.
- Obtained a (free) kit key from **Circle Console**.

## Step 1. Set up the project

This step shows you how to prepare your project and environment.

### 1.1. Set up your development environment

Create a new directory and install App Kit and its dependencies:

Shell

```
# Set up your directory and initialize a Node.js project
mkdir app-kit-quickstart-swap
cd app-kit-quickstart-swap
npm init -y

# Install App Kit and tools
npm install @circle-fin/app-kit @circle-fin/adapter-viem-v2 viem typescript ts
```

Only need to swap and want a lighter install than the full App Kit? Install the standalone swap package instead: `@circle-fin/swap-kit`

## 1.2. Configure TypeScript (optional)

This step is optional. It helps prevent missing types in your IDE or editor.

Create a `tsconfig.json` file:

Shell

```
npx tsc --init
```

Then, update the `tsconfig.json` file:

Shell

```
cat <<'EOF' > tsconfig.json
{
  "compilerOptions": {
    "target": "ESNext",
    "module": "ESNext",
    "moduleResolution": "bundler",
    "strict": true,
    "types": ["node"]
  }
}
EOF
```

### 1.3. Configure environment variables

Create an `.env` file in the project directory:

Shell

```
touch .env
```

Then, add your credentials. Replace `YOUR_PRIVATE_KEY` with the private key for your wallet and `YOUR_KIT_KEY` with the kit key from **Circle Console**:

.env

```
PRIVATE_KEY=YOUR_PRIVATE_KEY
KIT_KEY=YOUR_KIT_KEY
```

Edit `.env` files in your IDE or editor so credentials are not leaked to your shell history.

## Step 2. Perform the swap

This step shows you how to set up your script, execute a swap from USDC to EURC on Arc Testnet, and check the result.

### 2.1. Create the script

Create an `index.ts` file in the project directory and add the following code. This code swaps 1.00 USDC for EURC on Arc Testnet:

> Using other **tokens** or a different **blockchain**? Change the `tokenIn`, `tokenOut`, and `chain` values in `kit.swap()` and use an adapter for that chain.

TypeScript

```typescript
// Import App Kit and its dependencies
import { AppKit } from "@circle-fin/app-kit";
import { createViemAdapterFromPrivateKey } from "@circle-fin/adapter-viem-v2";
import { inspect } from "util";

// Initialize the SDK
const kit = new AppKit();

const swapUSDCtoEURC = async (): Promise<void> => {
  try {
    // Initialize the adapter
    const adapter = createViemAdapterFromPrivateKey({
      privateKey: process.env.PRIVATE_KEY as string,
    });

    console.log("---------------Starting Swapping---------------");

    const result = await kit.swap({
      from: { adapter, chain: "Arc_Testnet" }, // Adapter and blockchain for t
      tokenIn: "USDC", // Token to swap
      tokenOut: "EURC", // Token to receive
      amountIn: "1.00", // Amount of tokenIn to swap (human-readable)
      config: {
        kitKey: process.env.KIT_KEY as string, // Your kit key from the Circle
      },
    });

    console.log("RESULT", inspect(result, false, null, true));
  } catch (err) {
    console.log("ERROR", inspect(err, false, null, true));
  }
};

void swapUSDCtoEURC();
```

Customize your swaps to **collect a custom fee**, **set a slippage tolerance or stop limit**, or **get a pre-swap estimate**.

## 2.2. Run the script

Save the `index.ts` file and run the script in your terminal:

Shell

```
npx tsx --env-file=.env index.ts
```

## 2.3. Verify the transaction

After the script finishes, find the returned result object in the terminal output. Use the `explorerUrl` to verify the swap transaction and confirm the amount matches how much EURC was received.

The following code is an example of how the result of a successful swap might look in the terminal output. The values are used in this example only and are not a real transaction:

Shell

```
{
  tokenIn: 'USDC',
  tokenOut: 'EURC',
  chain: { chain: 'Arc_Testnet', isTestnet: true },
  amountIn: '1.00',
  amountOut: '0.99',
  fromAddress: '0x1234123412341234123412341234123412341234',
  toAddress: '0xabcdabcdabcdabcdabcdabcdabcdabcdabcdabcd',
  txHash: '0x78abb6a896e6b166925cae122b6ab2a6abd49ba23cbbd4749c99d5cccf205897'
  explorerUrl: 'https://testnet.arcscan.app/tx/0x78abb6a896e6b166925cae122b6ab
  fees: [ { token: 'USDC', amount: '0.001', type: 'provider' } ],
  config: {
    kitKey: 'KIT_KEY:fdd99fdbdb3c87b9b3f7b29d6cc17b6e:cc8777b4e419eb6dda9f0de9
  }
}
```

Was this page helpful? Yes No

Swap overview

Collect custom swap fees

Links

Network status

Blog

Community

Arc homepage

Resources

Terms of service

Privacy policy

Cookie policy

Powered by mintlify

By using the Arc Network Testnet, you agree to the Terms of Service and Privacy Policy.